

Computer Graphics

Detailed Notes — Curves, Surfaces & Spatial Data Structures

Contents

1. Convex Hull
2. Parametric Continuity Conditions
3. Geometric Continuity Conditions
4. Bezier Curves & Properties of Bezier Curves
5. B-Spline Curves
6. Beta-Splines
7. Rational Splines
8. Ray-Casting Methods
9. Octrees
10. BSP Trees

1. Convex Hull

The **convex hull** of a set of points S is the smallest convex set (polygon in 2D, polyhedron in 3D) that contains every point of S . A region is **convex** if, for any two points A and B inside the region, the entire straight line segment AB also lies inside the region. Intuitively, the convex hull is the shape formed by stretching an elastic band around all the outer points and letting it snap tight.

Convex Hull of a Point Set

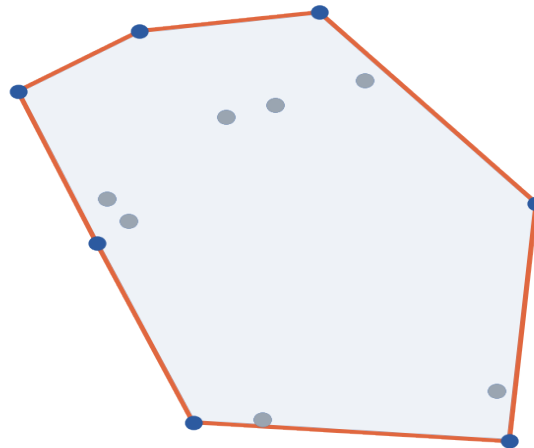


Fig 1.1 — Convex hull (red boundary) enclosing a set of scattered points.

Mathematical Definition

$$\text{Conv}(S) = \{ \sum \lambda_i P_i \mid P_i \in S, \lambda_i \geq 0, \sum \lambda_i = 1 \}$$

i.e. the convex hull is the set of all possible **convex combinations** of the points in S .

Why It Matters in Computer Graphics

Algorithms to Compute Convex Hull

Algorithm	Idea	Time Complexity
Brute Force	Test every pair of points as a potential edge; keep edges with all other points on one side.	$O(n^3)$
Graham's Scan	Sort points by polar angle from a pivot; scan and pop points causing a clockwise (non-left) turn.	$O(n \log n)$
Jarvis March (Gift Wrapping)	Starting from the leftmost point, repeatedly pick the next point on the convex hull by scanning all other points on one side, wrap around.	$O(n^2)$
Divide & Conquer	Recursively split point set, find hulls of halves, then merge (similar to merge sort).	$O(n \log n)$

Graham's Scan — Step by Step

2. Parametric Continuity Conditions

When multiple curve segments are pieced together to build a composite (piecewise) curve, we need conditions that describe how smoothly one segment joins the next. **Parametric continuity**, written C^n , is defined by matching the parametric derivatives of the two segments at the joint point exactly.

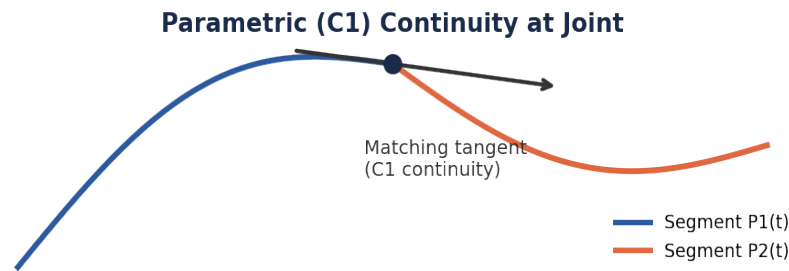


Fig 2.1 — Two curve segments joined with matching tangent vectors (C1 continuity).

Orders of Parametric Continuity

Order	Condition	Visual Meaning
C0	$P1(1) = P2(0)$	Segments share the same endpoint (simple positional join).
C1	C0, and $P1'(1) = P2'(0)$	Tangent vectors match exactly — same direction AND same magnitude
C2	C1, and $P1''(1) = P2''(0)$	Curvature vectors also match — the rate of change of the tangent is continuous

$$C0: P1(1) = P2(0) \quad C1: P1'(1) = P2'(0) \quad C2: P1''(1) = P2''(0)$$

Key Points

3. Geometric Continuity Conditions

Geometric continuity, written G^n , relaxes the strict requirement of parametric continuity. Instead of requiring derivative vectors to be exactly equal, it only requires them to point in the **same direction** (be scalar multiples of each other), regardless of magnitude. This gives designers more freedom to shape a curve while still keeping it visually smooth.

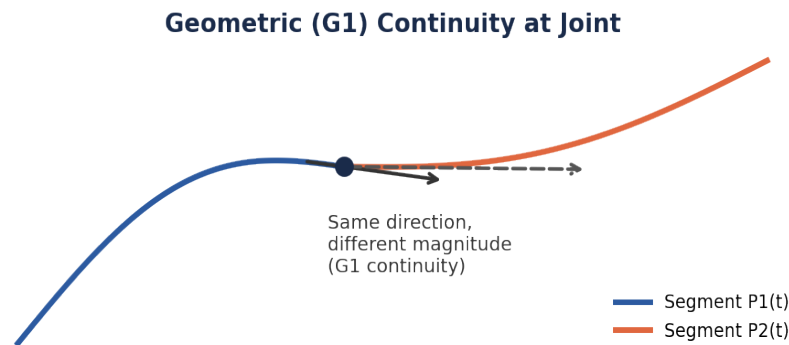


Fig 3.1 — G1 continuity: tangents share direction but differ in length.

Orders of Geometric Continuity

Order	Condition	Meaning
G0	$P1(1) = P2(0)$	Same as C0 — segments share an endpoint.
G1	Tangents at joint are parallel: $P1'(1) = k \cdot P2'(0)$, $k \geq 0$	Direction of travel matches, but speed (magnitude) can differ.
G2	G1, plus curvature is continuous	Rate of turning matches, giving a smooth, aesthetically pleasing join

Parametric vs Geometric Continuity

Aspect	Parametric Continuity (C_n)	Geometric Continuity (G_n)
Basis	Exact equality of derivative vectors	Proportionality (direction only) of derivative vectors
Strictness	Stricter / more rigid	More relaxed / flexible
Relationship	C_n continuity always implies G_n continuity	G_n continuity does NOT imply C_n
Typical Use	Animation paths, physics-based motion where speed matters	GUI/SVG/SVG surfaces design where only visual smoothness matters

Remember: $C1 \Rightarrow G1$ always, but $G1$ does NOT guarantee $C1$. Geometric continuity is generally preferred in curve/surface design software because it gives smoother-looking joins without forcing an artificial parametrization.

4. Bezier Curves & Properties of Bezier Curves

A **Bezier curve** is a parametric curve defined by a set of $n+1$ control points $P_0, P_1, \dots, P_n$ using Bernstein basis (blending) polynomials. It was developed by the French engineer **Pierre Bezier** at Renault for designing smooth automobile body surfaces.

General Bezier Equation

$$P(t) = \sum_{i=0}^n P_i \cdot BEZ_{i,n}(t), \quad 0 \leq t \leq 1$$

$$BEZ_{i,n}(t) = C(n,i) \cdot t^i \cdot (1-t)^{n-i} \quad \text{where} \quad C(n,i) = n! / (i! (n-i)!)$$

Cubic Bezier Curve ($n = 3$, four control points)

Cubic Bezier curves are the most widely used form (e.g. in fonts, vector graphics, and animation easing curves):

$$P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$$

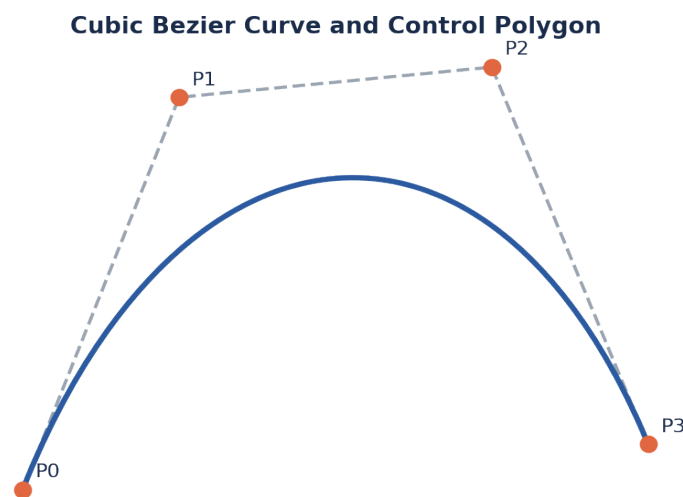


Fig 4.1 — A cubic Bezier curve with its four control points P_0 – P_3 and control polygon.

Properties of Bezier Curves

5. B-Spline Curves

A **B-Spline (Basis Spline) curve** generalizes the Bezier curve by using piecewise polynomial basis functions defined over a sequence of parameter values called a **knot vector**. This separates the curve's degree from the number of control points and gives the crucial advantage of **local control**.

B-Spline Equation

$$P(t) = \sum_{i=0}^n P_i \cdot N_{i,k}(t)$$

where k is the **order** of the curve (degree = $k-1$), P_i are the $n+1$ control points, and $N_{i,k}(t)$ are B-spline basis functions computed by the **Cox-de Boor recursion**:

$$N_{i,1}(t) = 1 \text{ if } t_i \leq t < t_{i+1}, \text{ else } 0$$

$$N_{i,k}(t) = [(t-t_i)/(t_{i+k-1}-t_i)] N_{i,k-1}(t) + [(t_{i+k}-t)/(t_{i+k}-t_{i+1})] N_{i+1,k-1}(t)$$

B-Spline: Local Control Property

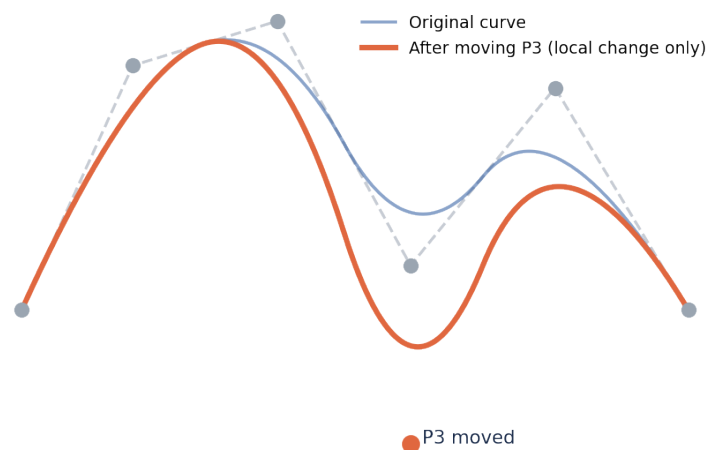


Fig 5.1 — Moving one control point (P_3) reshapes only a local part of the curve — the local control property.

Key Properties of B-Splines

Types of Knot Vectors

Type	Description
Uniform	Knot values are equally spaced (e.g. 0,1,2,3,4,...).
Non-uniform	Knot spacing is unequal, allowing more control over parametrization speed.
Open / Clamped	First and last knots repeated k times so the curve passes through the first and last control points (li
Periodic	Knots repeat cyclically to produce a smooth, closed curve.

6. Beta-Splines

A **Beta-spline** extends the B-spline by introducing two extra **shape parameters** — **bias** (β_1) and **tension** (β_2) — that let a designer reshape a curve without moving any control points, while still preserving smooth, controlled continuity across segments (called **β -continuity**, a generalized geometric continuity).

Shape Parameters

Parameter	Effect on Curve
Bias (β_1)	Controls how much the curve 'leans' toward the segment before or after a control point — asymmetry
Tension (β_2)	Controls how tightly the curve is pulled toward the straight-line control polygon — higher tension means tighter pull

Key Points

7. Rational Splines

A **rational spline** is defined as a ratio of two spline (polynomial) functions rather than a single polynomial. The most important example is the **NURBS** (Non-Uniform Rational B-Spline), in which every control point P_i is given an extra scalar **weight** h_i .

NURBS Equation

$$P(t) = \left[\sum_{i=0}^n h_i P_i N_{i,k}(t) \right] / \left[\sum_{i=0}^n h_i N_{i,k}(t) \right]$$

If all weights h_i are equal (e.g. all = 1), the equation reduces exactly to an ordinary (non-rational) B-spline curve.

Rational Spline (NURBS) Representing an Exact Circle

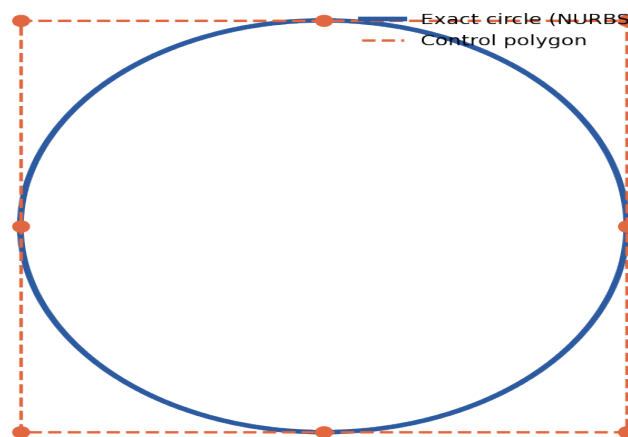


Fig 7.1 — A rational spline (NURBS) can represent a perfect circle exactly, unlike ordinary polynomial splines.

Why Rational Splines Matter

8. Ray-Casting Methods

Ray casting is a rendering technique for determining visible surfaces and their shading by firing a ray from the eye (camera) through every pixel of the image plane, and finding the closest scene object that the ray intersects.

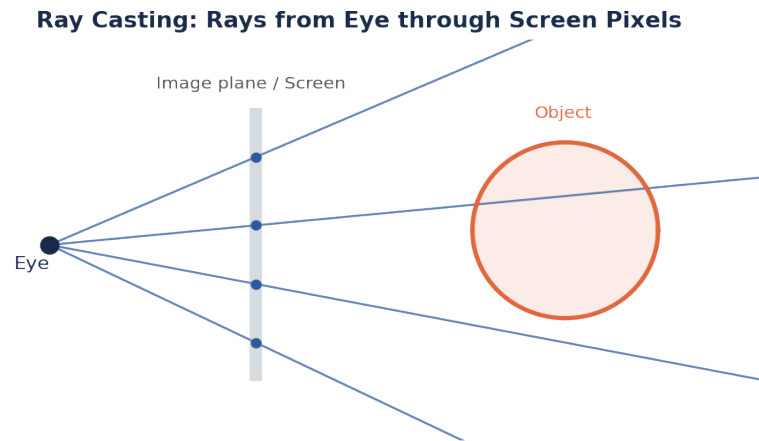


Fig 8.1 — Rays cast from the eye through each screen pixel toward scene objects.

Ray Equation

$$R(t) = O + t \cdot D, \quad t \geq 0$$

where O is the ray's origin (the eye position), D is the (normalized) ray direction through the pixel, and t is the distance parameter along the ray. The nearest object is the one with the smallest positive t at which $R(t)$ intersects an object's surface.

Algorithm Steps

Ray Casting vs Ray Tracing

Aspect	Ray Casting	Ray Tracing
Rays per pixel	One primary ray only	Primary ray + recursive secondary rays (reflection, refraction, etc.)
Realism	Basic shading, no reflections/shadows from other objects	Highly realistic, global illumination effects
Cost	Cheap, fast	Computationally expensive
Typical use	Volume rendering, older real-time games, QuickTime VR	Photorealistic rendering, offline film-quality images

Applications

9. Octrees

An **octree** is a hierarchical tree data structure used to partition 3D space by recursively subdividing a cubic region into **eight equal octants** (child cubes). It is the natural 3D extension of the 2D **quadtree**.

Octree Concept: Recursive Subdivision of Space (shown in 2D cross-section)

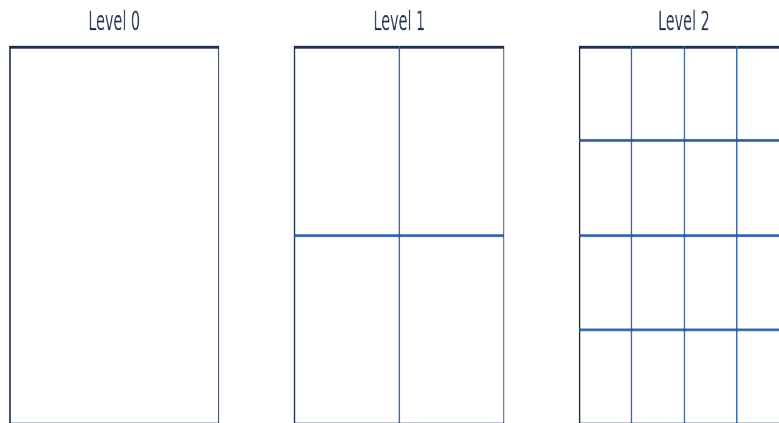


Fig 9.1 — Recursive subdivision of space at increasing levels of detail (shown as a 2D cross-section for clarity; a true octree splits a cube into 8 sub-cubes).

Construction Algorithm

Node Types

Node Type	Meaning
Full / Black node	Region is completely occupied by solid material.
Empty / White node	Region is completely empty space.
Partial / Grey node	Region is a mix of solid and empty — must be subdivided further (internal node with 8 children).

Applications

10. BSP Trees

A **Binary Space Partitioning (BSP) tree** recursively divides space into two half-spaces using hyperplanes (lines in 2D, planes in 3D), producing a binary tree that encodes the front/back spatial relationship of every object or polygon in a scene relative to those planes.

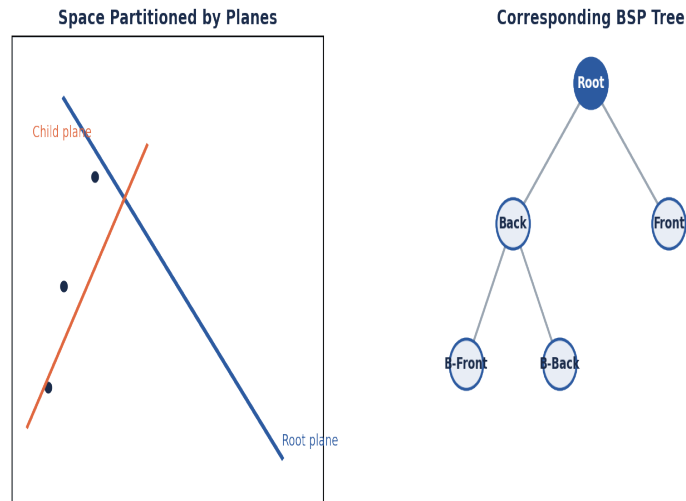


Fig 10.1 — A scene partitioned by planes (left) and the corresponding BSP tree structure (right).

Construction Algorithm

Rendering / Traversal (Painter's Algorithm via BSP)

Applications

Key advantage: Once a BSP tree is built, it can be traversed differently for any arbitrary viewpoint to yield correct back-to-front polygon order in $O(n)$ time — avoiding the need to re-sort polygons by depth every single frame, unlike the plain painter's algorithm.

— End of Notes —